

Your AI Working Guide

Keep building your websites with AI — and keep the AI on track.

Two playbooks in one: **Part 1** — how to keep working when one AI runs out of credits. **Part 2** — how to prompt any AI so it does exactly what you want and doesn't wander off.

Part 1 · Keep working when an AI runs out

First: your work is safe

Every project lives in **GitHub** and your live sites auto-publish from it. If an AI stops or hits a limit, **nothing is lost** — your sites keep running and you continue from any computer, with any AI.

The only rule that matters: **any change saved to GitHub publishes automatically**. So any tool that edits a file and saves to GitHub keeps you moving. You are never locked to one AI.

Which AI to use — in priority order

PLAN A **ChatGPT or Gemini + GitHub** — free / already have it. Ask it to write the change, then paste the file into your GitHub repo (open file → pencil → Commit). Publishes in ~1 minute. Your everyday go-to.

PLAN B **Cursor** (cursor.com, free tier) — feels closest to Claude; it edits the files for you. Open the project folder, chat, then commit + push. Alternatives: Windsurf (free), GitHub Copilot in VS Code.

PLAN C **No limits, pay-as-you-go** — OpenRouter (openrouter.ai, cheap models, cents per session) or the Anthropic Console. Use this if waiting keeps happening.

Handoff Prompt — paste into any new AI first

I'm continuing work on one of my websites. Before doing anything:

1. Read the README and main files so you understand this project.
2. Tell me in 2 lines what it is and how it's deployed.

My sites are plain HTML/CSS/JavaScript (no build step). They live on GitHub and auto-publish to the web. To publish a change, I commit it to GitHub.

Then help me with this change: [DESCRIBE WHAT YOU WANT].

Only change what I ask for, keep the existing design, and give me the complete updated file so I can paste it into GitHub.

Part 2 · How to drive the AI (without it going off the rails)

The 4-part prompt formula — use it every time

- **Context** — what the project is (or "read the repo first").
- **Goal** — the ONE thing you want, and what "done" looks like.
- **Boundaries** — what it must NOT change.
- **Output** — how you want it back (e.g. "the complete file").

Miss these and the AI guesses — and guessing is where it wanders off.

Copy-paste prompt template

```
CONTEXT: This is my [business] website. First read the project files.  
Keep the existing design and style.  
  
GOAL: [Exactly what you want]. It's done when [what you should see].  
  
BOUNDARIES: Only change [the specific part]. Do NOT touch anything else –  
keep the layout, colours, fonts and other sections exactly as they are.  
Don't add features I didn't ask for.  
  
OUTPUT: Show me your plan in 2–3 bullets first and wait for my "go".  
Then give me the complete updated file.
```

Guardrail phrases (keep these handy)

- "Only change [X]. Do not touch anything else."
- "Show me the plan first. Don't write code until I say go."
- "Keep the exact same design, fonts and colours."
- "Don't add features or pages I didn't ask for."
- "Give me the complete updated file, not snippets."
- "If you're not sure, ask me — don't guess."

Clear beats vague

Vague (AI wanders)	Clear (AI stays on track)
"Make the site better"	"On the hero, change the headline to '...'. Nothing else."
"Add a contact thing"	"In the footer, add a phone link showing 061 201 7287. Keep the rest."
"Fix the colours"	"Change the button from green to teal #2DE1C2. Nothing else."

Rule of thumb: if you can't picture the exact result, the AI can't either.

When it goes off track — recover fast

- "Stop. You changed things I didn't ask for. Undo those and only do [X]."
- "Go back to how it was, keep everything, just do [X]."
- Worst case: GitHub keeps every version — restore an older one from the file's `History`. You can't truly break it.

Golden rules: be specific · one change at a time · say what NOT to touch · plan first, build second · save to GitHub → check the live site.